

Role Overview

As the Lead Full-Stack Developer, this role focuses on the core architecture and implementation of the RMS (Restaurant Management System). The main areas of responsibility include backend development, authentication and authorization, business logic, real-time kitchen operations, admin portal functionality, and customer-facing website pages.

Key Responsibilities

- Designed and implemented the backend architecture using Node.js and Express.
- Developed REST API endpoints for restaurant operations.
- Implemented JWT-based authentication and role-based access control.
- Developed real-time communication for the Kitchen Display System using Socket.io.
- Built operational pages for the admin portal, including POS, kitchen, tables, billing, and analytics.
- Developed customer-facing pages for menu browsing, reservations, order tracking, and reviews.
- Integrated MongoDB for data storage and Cloudinary for menu image management.

1. Backend Development

Technology Stack: Node.js, Express.js, MongoDB, JWT, bcrypt, Socket.io, Multer, Cloudinary, Helmet, and CORS.

The backend serves as the central processing layer of RMS. It handles authentication, restaurant operations, database interactions, API requests, and real-time updates.

1.1 Core Server and Authentication

File	Purpose
server/index.js	Main server entry point. Configures Express, connects to MongoDB, registers routes, initializes Socket.io, and applies security middleware such as Helmet and CORS.
server/middleware/auth.js	Implements JWT verification for protected API requests and role-based access control for restricted operations.

<code>server/controllers/authController.js</code>	Handles staff login, verifies credentials using bcrypt, generates signed JWTs, and returns user profile information.
---	--

1.2 Backend Controllers

Controllers contain the business logic for the application's main modules.

File	Purpose
<code>server/controllers/orderController.js</code>	Handles order creation, order retrieval, order status updates, table-related orders, and Socket.io events for kitchen updates.
<code>server/controllers/menuController.js</code>	Provides menu item CRUD operations, image uploads through Multer and Cloudinary, category filtering, availability management, and pricing updates.
<code>server/controllers/tableController.js</code>	Manages table availability, occupancy, reservations, table status updates, and floor-map data.
<code>server/controllers/analyticsController.js</code>	Uses MongoDB aggregation pipelines to calculate revenue, order volume, and top-selling items for dashboards and reports.
<code>server/controllers/reservationController.js</code>	Handles reservation creation, retrieval, updates, cancellations, date-slot validation, and occasion information.
<code>server/controllers/paymentController.js</code>	Creates payment records linked to orders, records payment methods, and updates invoice payment status.
<code>server/controllers/staffController.js</code>	Provides staff account management, including user creation, password hashing, role assignment, and administrative CRUD operations.
<code>server/controllers/branchController.js</code>	Manages restaurant branch information, including names, addresses, and operating hours.

<code>server/controllers/inventoryController.js</code>	Tracks ingredient stock levels and supports low-stock alerts based on configured thresholds.
<code>server/controllers/reviewController.js</code>	Accepts customer reviews and stores ratings and comments associated with menu items.

1.3 API Routes

Directory: `server/routes/`

The route files connect incoming HTTP requests to their corresponding controllers.

Route File	Responsibility
<code>analyticsRoutes.js</code>	Dashboard and analytics endpoints under <code>/api/analytics/</code> .
<code>auth.js</code>	Authentication endpoints, including <code>POST /api/auth/login</code> .
<code>branches.js</code>	Restaurant branch endpoints under <code>/api/branches/</code> .
<code>inventory.js</code>	Inventory management endpoints under <code>/api/inventory/</code> .
<code>menu.js</code>	Menu management endpoints under <code>/api/menu/</code> .
<code>orders.js</code>	Order management endpoints under <code>/api/orders/</code> .
<code>payments.js</code>	Payment and billing endpoints under <code>/api/payments/</code> .
<code>public.js</code>	Public endpoints under <code>/api/public/</code> for customer-facing features.
<code>reservations.js</code>	Reservation endpoints under <code>/api/reservations/</code> .
<code>reviews.js</code>	Customer review endpoints under <code>/api/reviews/</code> .
<code>staff.js</code>	Staff management endpoints under <code>/api/staff/</code> .
<code>tables.js</code>	Table management endpoints under <code>/api/tables/</code> .

1.4 Real-Time Kitchen Display System

File: `server/sockets/orderSocket.js`

The Kitchen Display System uses Socket.io to deliver order updates to connected kitchen screens without requiring a page refresh.

Key responsibilities:

- Establishes Socket.io communication for kitchen operations.
- Supports real-time delivery of incoming orders to the kitchen interface.
- Broadcasts relevant order status changes to connected clients.
- Helps kitchen staff and service staff stay synchronized during order preparation.

Order Workflow

POS Order Creation

|

v

Backend Order API

|

v

MongoDB Order Storage

|

v

Socket.io Event

|

v

Kitchen Display System

|

v

Order Status Update

|

v

Admin and Customer Interfaces

The exact events and status transitions depend on the implementation in the project source code.

2. Admin Portal Development

Directory: `client/`

The admin portal provides staff with an interface for managing restaurant operations, including orders, kitchen workflows, tables, menu items, billing, reservations, and reports.

2.1 Application Routing and Access Control

File: `client/src/App.jsx`

Responsibilities:

- Defines routes for the admin application.
- Implements protected routes through `PrivateRoute`.
- Restricts access to authenticated users.
- Applies role-based navigation and redirection for different staff roles.

2.2 Core Operational Pages

File	Purpose
<code>client/src/pages/Kitchen/Kitchen.jsx</code>	Provides a real-time Kitchen Display System with incoming order cards, item lists, timers, and order status controls. Connects to Socket.io for live updates.
<code>client/src/pages/Orders/Orders.jsx</code>	Provides the Point-of-Sale interface for browsing menu items, building orders, assigning tables, adding special notes, and managing active orders.
<code>client/src/pages/Tables/Tables.jsx</code>	Displays an interactive restaurant floor map with color-coded table statuses. Allows staff to inspect table orders and update table status.
<code>client/src/pages/Menu/Menu.jsx</code>	Provides menu management features for creating, editing, and deleting items, uploading images, setting prices and categories, and changing item availability.

2.3 Dashboard, Billing, and Reports

File	Purpose
<code>client/src/pages/Dashboard/Dashboard.jsx</code>	Displays key performance indicators, revenue trends, order statistics, table occupancy, popular menu items, and quick actions. Uses Recharts for visualizations.

<code>client/src/pages/Billing/Billing.jsx</code>	Supports invoice preparation, subtotal and tax calculations, printable bills, payment-method selection, and payment status updates.
<code>client/src/pages/Reports/Reports.jsx</code>	Displays revenue trends, order volume, and top-performing menu items through charts powered by Recharts.
<code>client/src/pages/Reservations/Reservations.jsx</code>	Provides reservation management, including viewing, creating, updating, and cancelling bookings, along with date, time-slot, and occasion information.

3. Customer Website Development

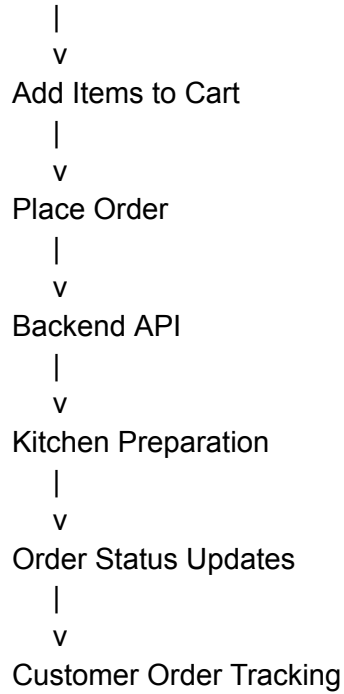
Directory: `customer/src/pages/`

The customer website allows visitors to explore the restaurant, browse menu items, make reservations, track orders, and submit reviews.

File	Purpose
<code>customer/src/pages/Landing.jsx</code>	Public homepage featuring a hero section, featured dishes, restaurant information, and calls to action for menu browsing.
<code>customer/src/pages/Menu.jsx</code>	Displays the public menu with category filters, dish descriptions, prices, dietary tags, and cart integration.
<code>customer/src/pages/Reservations.jsx</code>	Provides a reservation form for selecting a date, time slot, party size, and occasion.
<code>customer/src/pages/Tracking.jsx</code>	Allows customers to view the status of their placed orders.
<code>customer/src/pages/Review.jsx</code>	Provides a form for submitting ratings and comments about the dining experience.

Customer Order Workflow

Browse Menu



4. Key Technologies

Technology	Purpose
Node.js	JavaScript runtime for backend development.
Express.js	Backend framework for routing and API development.
MongoDB	Database for storing application data.
React	Library for building the admin and customer interfaces.
Socket.io	Real-time communication for kitchen and order updates.
JWT	Token-based authentication.
bcrypt	Password hashing and credential verification.
Axios	HTTP communication between frontend and backend services.
Zustand	Global state management in the frontend.
Recharts	Dashboard and analytics visualizations.

Cloudinary	Menu image storage and management.
Multer	Handling uploaded files.
Helmet	Security-related HTTP headers.
CORS	Controls cross-origin requests.

5. Suggested Git Commit Plan

Use the following commit structure when the listed files are ready and belong to your contribution.

Commit 1 — Backend Development

Files:

- `server/index.js`
- `server/middleware/`
- `server/controllers/`
- `server/routes/`
- `server/sockets/`

Commit message:

feat: implement backend APIs, authentication, order lifecycle, and real-time events

Commit 2 — Admin Frontend Core

Files:

- `client/src/App.jsx`
- `client/src/pages/Kitchen/`
- `client/src/pages/Orders/`
- `client/src/pages/Tables/`

Commit message:

feat: implement kitchen display, POS order management, and interactive floor map

Commit 3 — Admin Frontend Supporting Modules

Files:

- `client/src/pages/Dashboard/`
- `client/src/pages/Billing/`
- `client/src/pages/Reports/`
- `client/src/pages/Reservations/`
- `client/src/pages/Menu/`

Commit message:

feat: add dashboard analytics, billing, reports, reservations, and menu management

Commit 4 — Customer Website**Files:**

- `customer/src/pages/`

Commit message:

feat: implement customer landing page, menu, reservations, tracking, and reviews

6. Before Publishing to GitHub

- Verify that all documented files exist in the repository.
- Confirm that the setup instructions in the main `README.md` are accurate.
- Add required environment-variable names to an `.env.example` file without including actual secret values.
- Ensure `.env` files, passwords, API keys, JWTs, and production credentials are excluded from version control.
- Verify that authentication and role-based access checks are implemented correctly.
- Confirm that menu images and other third-party assets are permitted for publication.
- Test the backend and both frontend applications before presenting the project as complete.

Contribution Summary

This contribution covers the core full-stack implementation of RMS, including backend architecture, REST APIs, authentication and authorization, order and payment workflows, real-time kitchen updates, admin operational pages, and customer-facing website features.

